

PSP 0.1 — Process Improvement Proposal (PIP) Template

Nombre: Bridget Mendez Fecha: 9-12-25 Proyecto: Busqueda Versión de PSP: 5.0

1. Descripción del Problema

Durante el desarrollo de Program 6, tuve varios problemas relacionados con la **precisión numérica** y las **pruebas**:

Primero implementé la lógica de integración (SimpsonIntegration) y después la búsqueda (Busqueda), pero **no definí desde el inicio un criterio claro de precisión** (tolerancia para p y para la integral) ni cómo iba a validar los resultados.

Esto provocó que los primeros resultados del programa fueran **difíciles de interpretar**:

- En los casos 1 y 2, los valores de x eran casi iguales a los de la tabla, pero en el caso 3 ($p = 0.495$, $dof = 4$) el x se veía “raro” comparado con el valor de referencia.
- Me generó la duda de si había un **defecto en los cálculos**, cuando en realidad el problema era la tolerancia y lo plana que es la curva t-Student en la cola.

Tuve que hacer varios ajustes “de prueba y error” en:

- `toleranciaP` dentro de `Busqueda`,
- `errorSimpson` dentro de `SimpsonIntegration`,
- y también en la lógica de pruebas con Excel, hasta entender que el criterio real de aceptación debía ser el error en p y no tanto la diferencia en x .

2. Propuesta de Mejora Para futuros programas que usen **cálculo numérico** (integración, búsqueda, métodos iterativos), propongo lo siguiente:

1. **Definir explícitamente, desde la fase de diseño (PSP – Design / Logic Spec), los criterios de precisión y aceptación**, incluyendo:

- Tolerancia máxima para el valor objetivo (por ejemplo, $|p(x) - p_{\text{objetivo}}| \leq 1E-7$).
- Tolerancia de la integración numérica (errorSimpson o equivalente).
- Si la comparación principal será sobre la variable de salida (x) o sobre la función objetivo (p).

2. Diseñar y documentar una tabla de casos de prueba antes de codificar, con:

- Entradas (p, dof).
- Salidas esperadas (x de tabla).
- Tolerancias aceptables (para p y para x).
- Criterio de "APROBADO / FALLA".

3. Integrar esta tabla de pruebas en una hoja de Excel desde el principio, en vez de hacerlo después, para:

- Registrar lo que debería dar el programa.
- Registrar lo que realmente da el programa.
- Dejar fórmulas de error y de resultado (OK / FALLA).

4. Reflejar los parámetros clave (tolerancias) en el código como constantes bien nombradas en vez de poner los valores "duros" (magic numbers) dentro de los métodos.

3. Justificación Tener los criterios de precisión definidos desde el principio **reduce la ambigüedad** cuando los resultados no coinciden exactamente con los valores tabulares.
- En Program 6, el caso 3 dio la impresión de ser un error de implementación, cuando en realidad el cálculo era correcto dentro de la tolerancia, pero esto **no estaba claro en el diseño ni en las pruebas**.
 - Una hoja de pruebas preparada desde la fase de diseño habría permitido:

- Detectar antes que el error relevante era el de **p**,
- Entender que, en regiones donde la función es muy plana, pequeños errores de **p** se traducen en diferencias más grandes en **x**.
- Documentar y usar constantes para las tolerancias (por ejemplo `FINAL_P_TOLERANCE`, `SIMPSON_TOLERANCE`) haría el código:
 - Más legible.
 - Más fácil de ajustar.
 - Más fácil de revisar durante Code Review.

En resumen, la mejora propuesta reduce **retrabajo, confusión y tiempos de depuración** en programas que usan métodos numéricos iterativos.

4. Plan de Implementación

null

5. Resultados Esperados

Los ajustes de tolerancias dejarán de ser “prueba y error” informal y se convertirán en parte del proceso documentado (Design → Code → Test → PIP).

Los futuros programas que utilicen Simpson, búsqueda o cualquier método numérico iterativo se beneficiarán directamente de esta experiencia.